Homework #3

Aaron Chen / aaronyc

A1. Conceptual Questions

(a) False. Neural networks are non-convex (e.g. multilayer cross-entropy loss). Gradient descent can still be run, but it only has stronger optimization guarantees for convex objectives, so it is not guaranteed to find the global optimum.

(b) False. Initializing all weights to zero fails to break symmetry: neurons in the same layer get the same gradients and remain identical during training. Random initialization is preferred because it lets different neurons learn different features.

(c) True. Examples of non-linear activation functions: Sigmoid, ReLU, tanh.

(d) False. Backpropagation reuses intermediate values from the forward pass and computes all gradients efficiently with the chain rule. The backward pass is usually only a constant-factor more expensive than the forward pass, not prohibitively larger.

(e) False. Neural networks are flexible, but they are not always the best choice. For example, in settings where interpretability is important, simpler models such as linear or logistic regression may be preferable.

A2. Kernels: RBF Feature Map

$$\begin{aligned}
\phi(x) \cdot \phi(x') &= \sum_{i=0}^{\infty} \left(\frac{1}{\sqrt{i!}} e^{-x^2/2} x^i \right) \left(\frac{1}{\sqrt{i!}} e^{-x'^2/2} x'^i \right) \\
&= e^{-x^2/2} e^{-x'^2/2} \sum_{i=0}^{\infty} \frac{(xx')^i}{i!} \\
&= e^{-x^2/2} e^{-x'^2/2} e^{xx'} \\
&= e^{-\frac{x^2}{2} - \frac{x'^2}{2} + xx'} \\
&= e^{-\frac{x^2 - 2xx' + x'^2}{2}} \\
&= e^{-\frac{(x-x')^2}{2}} \\
&= K(x, x')
\end{aligned}$$

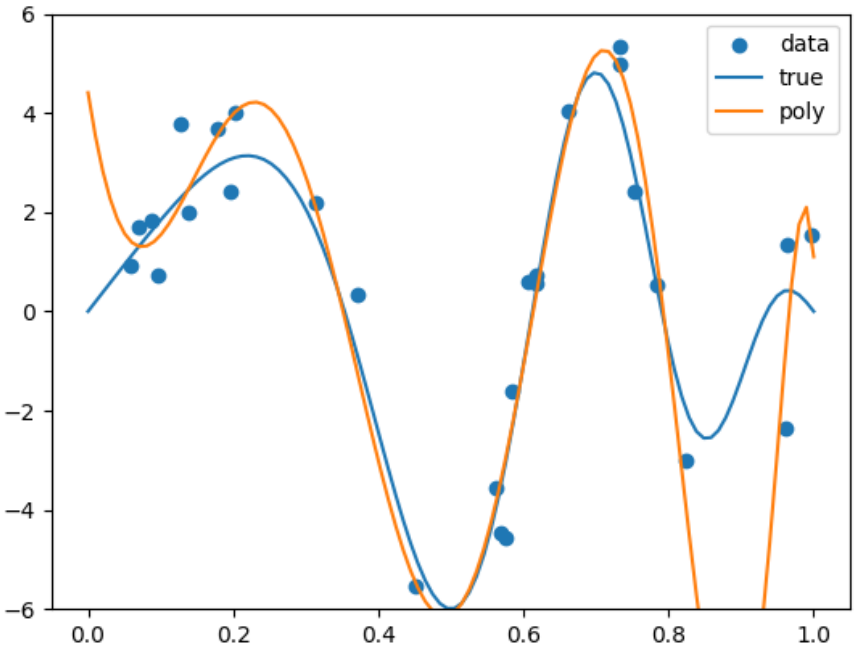
$$K(x, x') = e^{-\frac{(x-x')^2}{2}}$$

A3. Kernel Ridge Regression

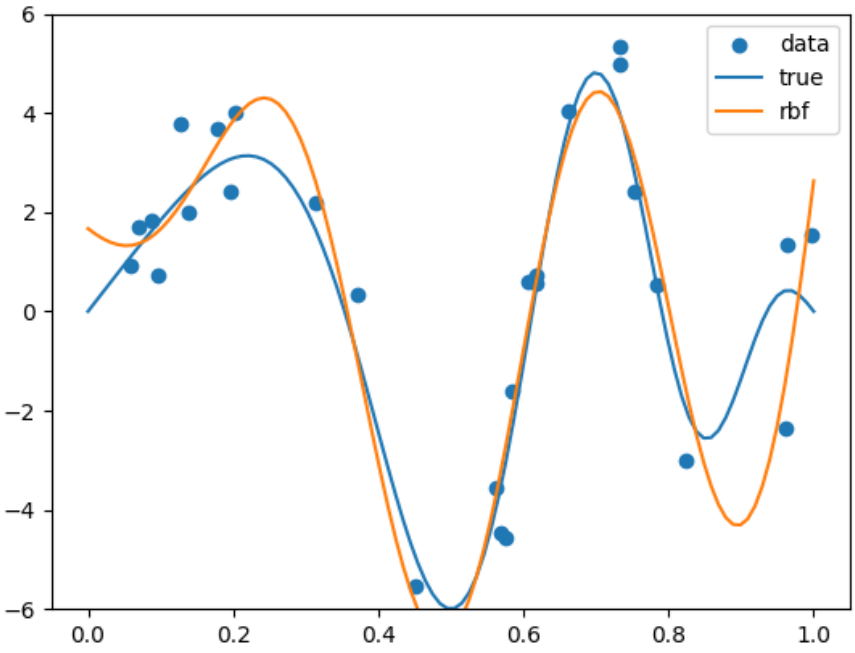
(a) The selected hyperparameters are:

Kernel	d	γ	λ
Polynomial	16	–	$4.281332398719396 \times 10^{-5}$
RBF	–	10.541635373659384	0.00206913808111479

(b) Polynomial kernel:

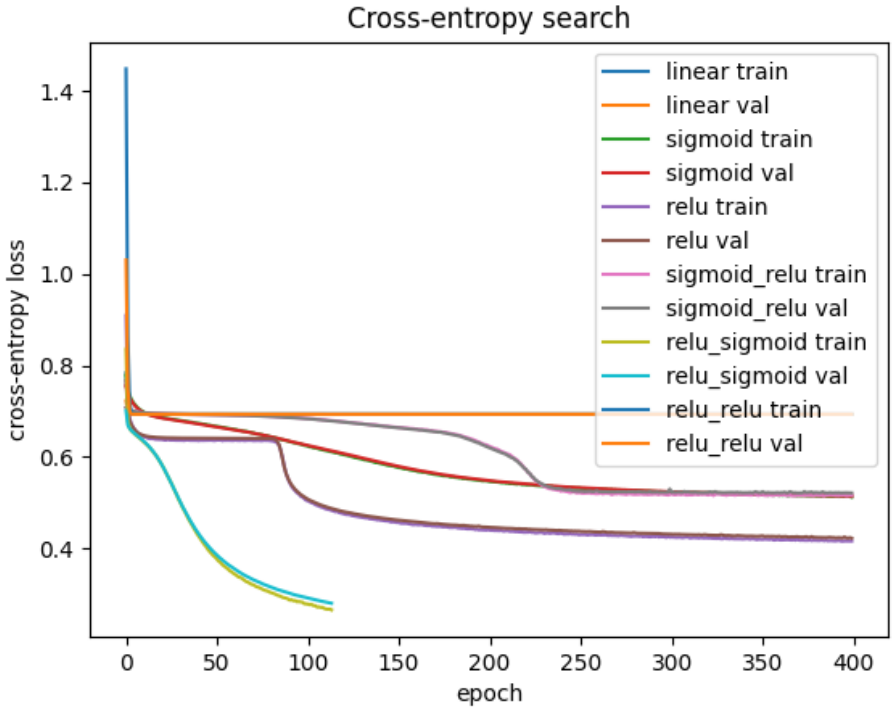


RBF kernel:

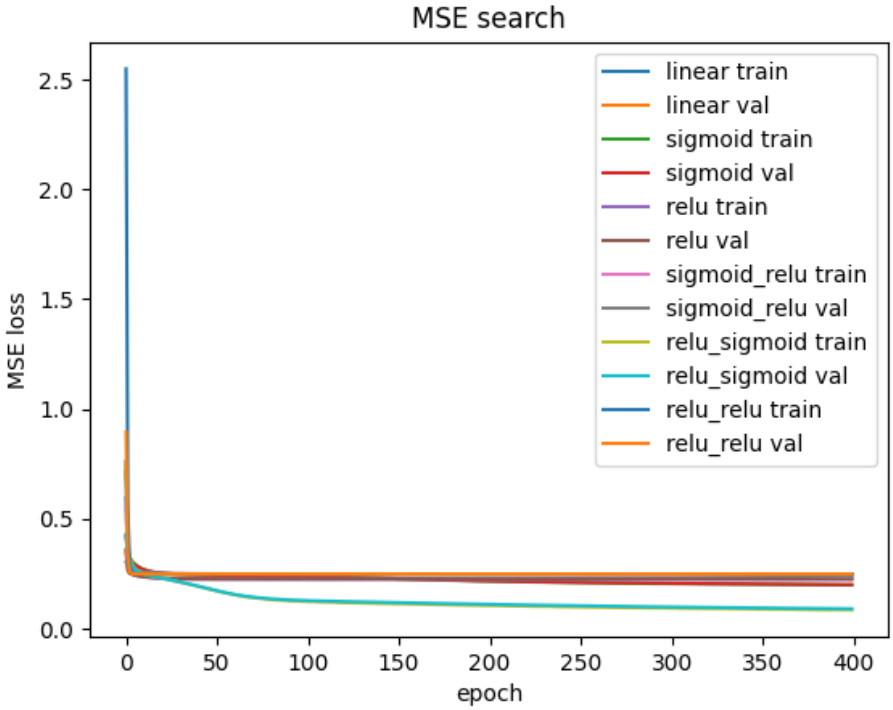


A4. Introduction to PyTorch

(b) Cross-entropy loss:

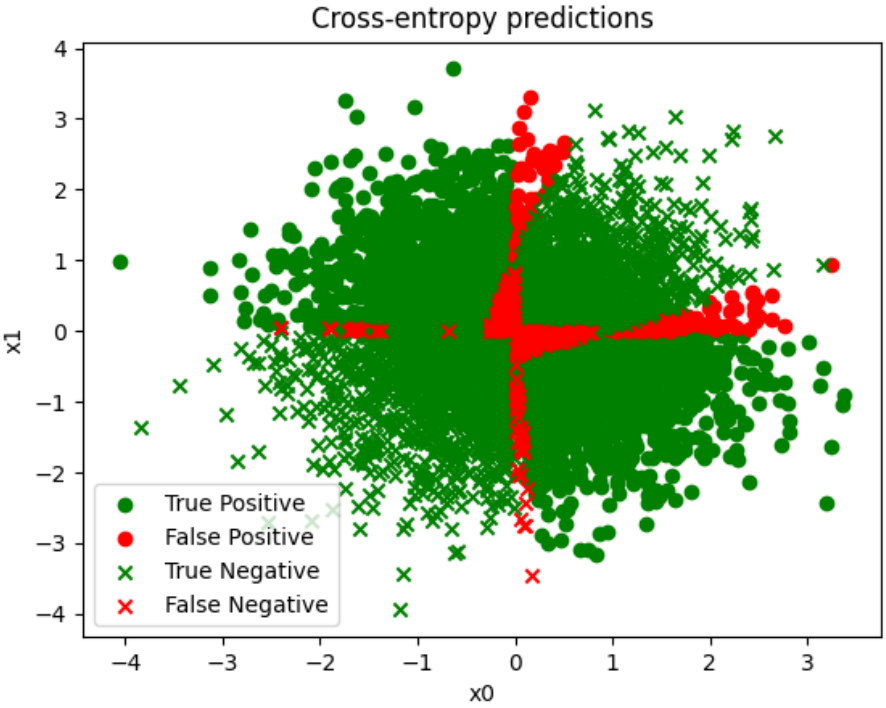


Mean squared error loss:



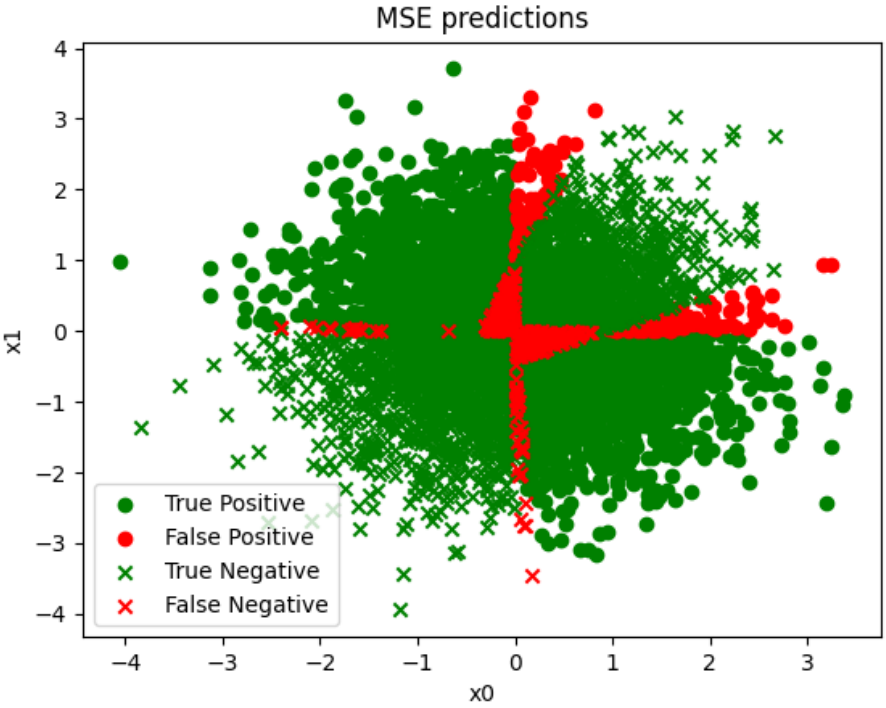
(c) For cross-entropy loss, the best-performing architecture is ReLU then Sigmoid.

Test accuracy = 0.9184



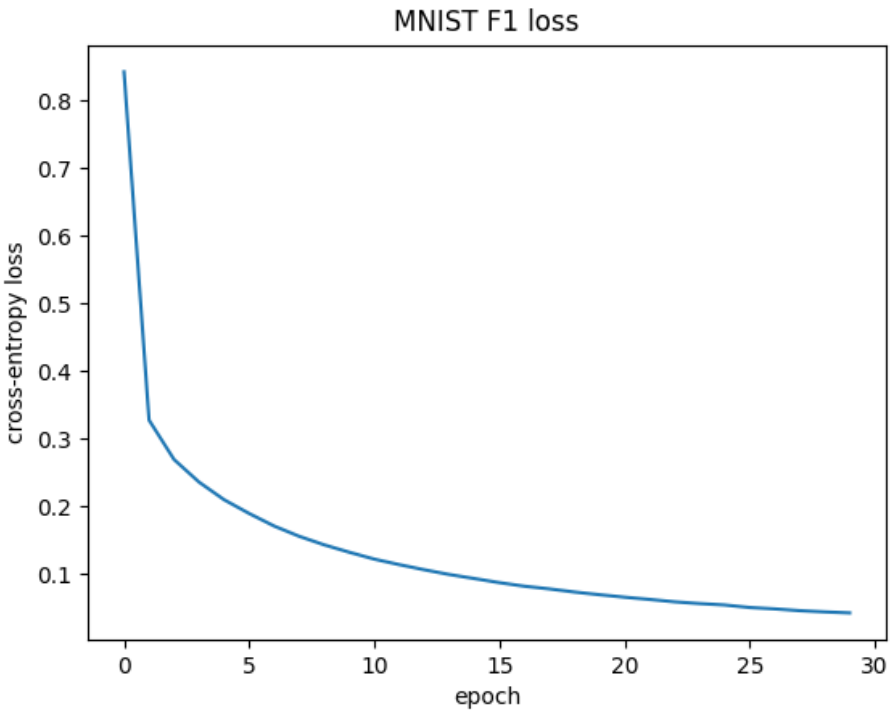
For mean squared error loss, the best-performing architecture is ReLU then Sigmoid.

Test accuracy = 0.9092



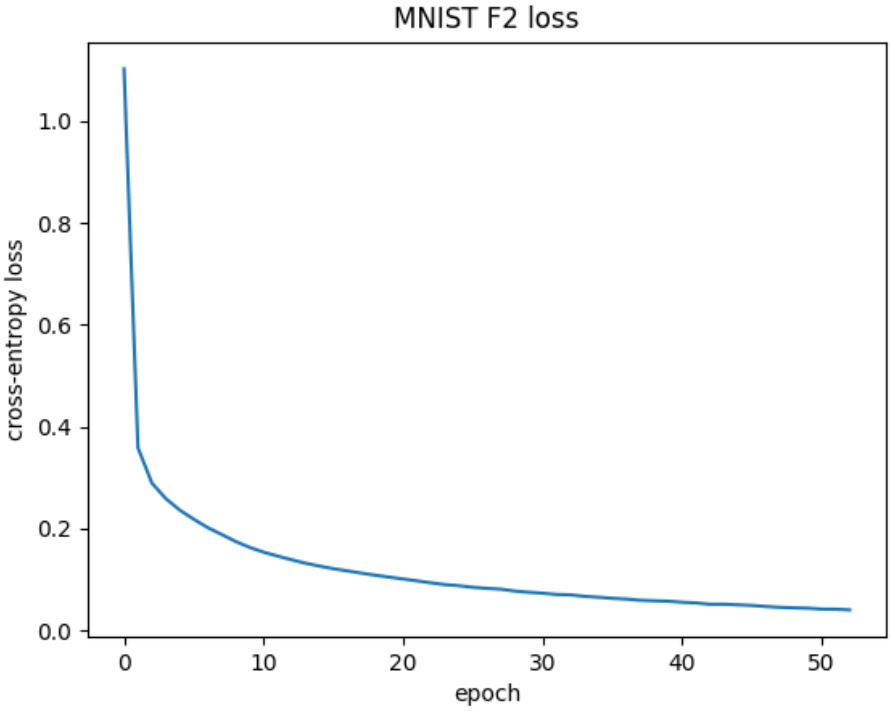
A5. Neural Networks for MNIST

(a) Wide shallow network:



Test accuracy = 0.9736
Test loss = 0.0871247056722641

(b) Narrow deeper network:



Test accuracy = 0.965
Test loss = 0.13156537783145905

(c)

#parameters for wide shallow network = 50890
#parameters for narrow deeper network = 26506

The wide shallow network has more parameters and slightly better test accuracy. The narrow deeper network uses about half as many parameters, but its accuracy is only slightly lower.

A6. Multinomial Logistic Regression

(a) For one example (x_i, y_i),

$$L_i(W) = -\log p(y_i \mid x_i; W)$$

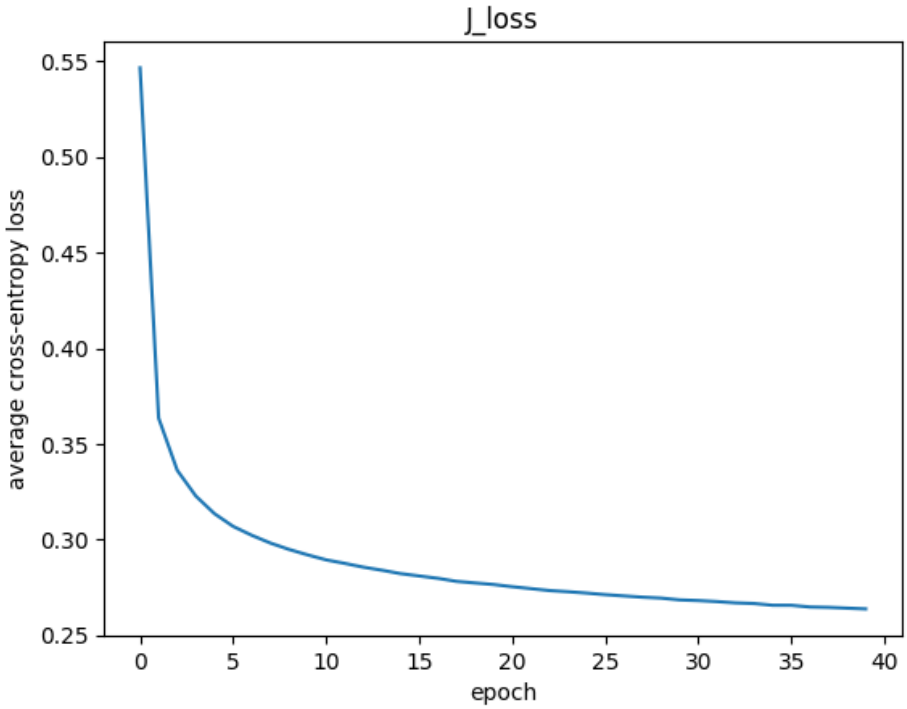
$$\begin{aligned} L_i(W) &= -\log\left(\frac{\exp(W_{y_i}^\top x_i)}{\sum_{\ell=1}^k \exp(W_\ell^\top x_i)}\right) \\ &= -W_{y_i}^\top x_i + \log\left(\sum_{\ell=1}^k \exp(W_\ell^\top x_i)\right) \end{aligned}$$

$$\begin{aligned} \nabla_{W_j} L_i(W) &= -\mathbf{1}\{j = y_i\}x_i + \frac{\exp(W_j^\top x_i)}{\sum_{\ell=1}^k \exp(W_\ell^\top x_i)} x_i \\ &= (p(y = j \mid x_i; W) - \mathbf{1}\{j = y_i\})x_i \end{aligned}$$

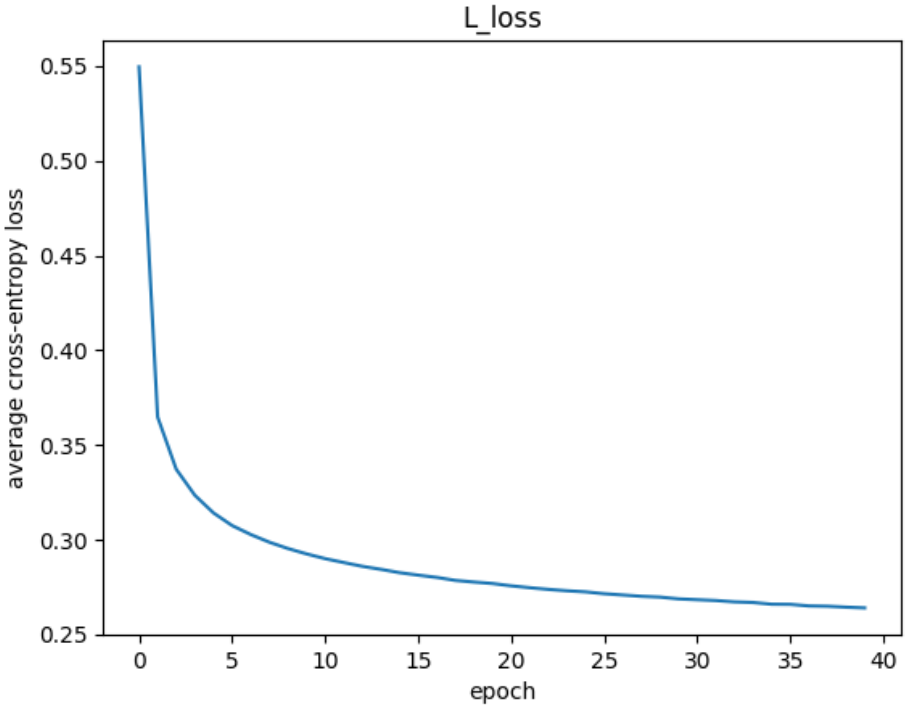
(b) The results are:

Loss	Training accuracy	Test accuracy
`J_loss`	0.9290	0.9245
`L_loss`	0.9280	0.9233

`J_loss` plots:



`L_loss` plots:


L_loss
average cross-entropy loss
0.55
0.50
0.45
0.40
0.35
0.30
0.25
0
5
10
15
20
25
30
35
40
epoch